

Generating Enhanced Slow Motion Frame from Blurred Video Segments

Sai Suryateja Sakuru, Ishan Choudhary, Aniket Wagde, and Joshua Realubit

Project Submission for CS415 Computer Vision

University of Illinois at Chicago

December 11, 2021

Abstract

Video deblurring is a challenging problem in Computer Vision which aims to recover sharp frames from blurry frames in a video. The blur in the data set is caused by camera shake, depth variation and object motion. We present a new method for non-uniform motion deblurring by applying a multi-scale convolutional neural network and Gaussian pyramid sharpening. Our method is trained end-to-end on a large data set comprising of realistic blurred images along with their corresponding ground truth image. Furthermore, we apply dense Gunnar-Farneback optical flow based selective sharpening using to refine objects in motion in the sharpened image. Our method achieves a high accuracy and is robust to variations in the blur kernel.

Keywords

Convolutional neural network, deep learning, motion deblurring, Optical Flow

1 Introduction

Videos captured under long exposure times often undergo shaking and object motion which degrades image quality and manifests in the form of blurring. This degradation is an undesirable effect which limits the quality of a video and renders the frames less usable for the photographer. By applying modern deep learning computer vision methods, we reverse and recover the sharp original image. This has applications in various domains such as photography, cinematography and video conferencing. In addition, the sharpened image can improve increased performance on various applications such as Image Classification, Object Detection and Face Recognition.

In contrast to previous works which focus on

blurring caused by simple factors such as translation or rotation of the camera. Our work focuses on complex non-uniform blurs, which are a result of camera shaking, depth variation and object motion.

According to Zhang et al [1] and Nah et al [2], motion blur can be considered to be a temporal integration of multiple sharp frame and can be formulated as:

$$I_B = g(\frac{1}{T} \int_0^1 I_S(t) dt)$$

Where I_B is the blurred image, T is the exposure duration, $I_S(t)$ is the sharp frame at time t and $g()$ represents the Camera Response Function (CRF).

Another approach is to formulate blurred non-uniform motion as a linear transformation by the following equation:

$$I_B = I_S K + N$$

Where I_B is the blurred image, I_S is the sharpened image, K is the blur kernel matrix and N is the additive noise. The blur kernel matrix K is applied to the sharp image by matrix multiplication. The aim is to find I_S given I_B assuming K and N are unknown.

Furthermore, Kim and Lee [3] proposed an approach to approximate the latent image and local linear motion jointly.

Although all these approaches are promising, they are still not adequately accurate to model blur especially in cases of occlusions and abrupt motions in frames. Most approaches require hand-crafting priors which is a difficult task.

In the recent years, neural networks have been very successful in image restoration tasks, especially Convolution Neural Networks, which have shown great promise in deblurring and image

recovery. The priors are implicitly learned by the network which are then used to predict the sharpened image.

In our proposed work, we use a multi-scale CNN [2] (Fig. 1) wherein all parameters are learned by the model and a direct prediction of the deblurred image is made and combine that with an optical flow estimation based method to further refine and sharpen the image. We achieve the generation of the sharpened frames through a Generative Adversarial Network (GAN) in which we use a ResNet that allows the gradient to propagate to deeper layers using skip-connections that allow direct connections which skip some layers. In our Generative Adversarial Network, the generator uses transpose convolution layer to generate an image while the discriminator labels the image as authentic or inauthentic. Both these work in conjunction to create the most realistic frames. For the loss function for the network, we apply a multi-scale loss in addition to an adversarial loss which is minimized using the ADAM optimizer. We use a data set in which blurred images are prepared by averaging a sequence of sharp frames in dynamic setting.

Furthermore, for further refinement and focus inside frames we have created a novel method that we term Optical Flow based Selective Sharpening wherein we use Gunnar-Farneback optical flow to determine portions of motion within an image and mask those regions with the sharpened image to obtain a selectively sharpened image with a focus effect on the objects of motion.

Optical flow measures change in pixel intensity over time with the assumption of constancy in brightness of an object point over time and similarity in movement of nearby points in the image plane. The optical flow equation is given as:

$$\frac{dI}{dx}u + \frac{dI}{dy}v + \frac{dI}{dt} = 0$$

In the equation, $\frac{dI}{dx}$ and $\frac{dI}{dy}$ are the image gradients along the horizontal and vertical axes respectively and $\frac{dI}{dt}$ is the gradient along the time. In our work, we use dense optical flow, in which we compute the optical flow for every pixel of the frame and estimate the magnitude and direction of optical flow from an array of the flow vectors $\frac{dx}{dt}$ and $\frac{dy}{dt}$.

2 Materials & Methods

GoPro Dataset

The GoPro data set used for the task of deblurring consists of 3,214 pairs of blurred

and sharp images of resolution 1280x720 out of which, 2,103 paired images are used as training set and the remaining 1,111 paired images as test set. The ground truth sharp images are obtained from the high speed camera for the same instance for which a blurry image is captured. The GoPro dataset is created using the GOPRO4 Hero Black camera. The blurs of different strengths were created by taking 240 fps videos with GOPRO camera and then averaging varying numbers of successive latent frames. The data set is publicly available.

Proposed Model

In the model, we are using finer scale image deblurring aided by coarser scale features. In order to preserve the information at the finer level level, we use Gaussian pyramid scaling. In Gaussian pyramid, the subsequent images are weighted down and scaled down using Gaussian blur. Each pixel which contains a local average is connected to the neighboring pixel on a lower level of the pyramid.

Model Architecture

We are using multi-scale CNN that stores latent images without assuming any restricted blur kernel model. The multi-scale architecture is designed to replicate the behavior of coarse to fine optimization methods. So, in this method, the architecture is independent of the kernels. We adopted kernel free methods in both blur data set generation and latent image estimation because of the following three reasons:

- simple kernel convolution cannot model challenging problems such as depth variations.
- kernel estimation process is very sensitive to noise and saturation.
- huge amount of memory and computation speed is required if we have to find spatially varying kernel for every pixel.

We are also using Residual neural network (ResNet) [2] as a building block (Fig. 2) for the proposed architecture. So, ResNet enables deeper flow compared to a traditional CNN. We observed that if we remove ReLU (Rectified Linear Unit) before the output boosts the convergence speed at the training time.

We are using $K = 3$ scales as default for the ResNet. So, we are having the input and output Gaussian pyramid resolution as 256x256, 128x128 and 64x64. [2] The scale ratio between the scales of the Gaussian pyramids is taken as 0.5. The filter size for the convolutional layer be 5x5 (Table 1).

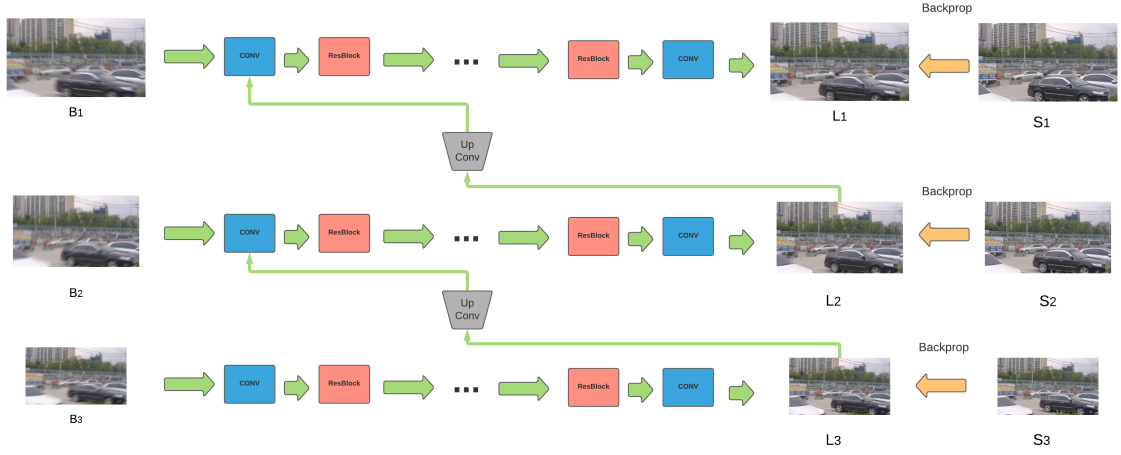


Figure 1: Multi-scale CNN

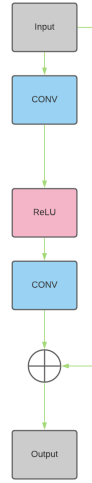


Figure 2: Multi-scale CNN

In the generator as given in Figure 1, the blurry image at a lower scale passed through convolution layers with residual blocks to generate a latent image. The predicted latent image is then compared to the ground truth sharp image to generate a loss. We compute the gradients through back-propagation and update the weights using ADAM optimizer. The predicted latent image is then up-sampled and passed to another convolution layer with a higher scaled version of the image and the same process is repeated until we obtain the highest scale. The discriminator then determines the authenticity of the highest scale or ground truth image. Both the generator and discriminator are jointly trained using gradient generated through the adversarial and multi-scale content loss.

We formulate the adversarial loss as:

$$L_{adv}(G, D, B, S) = E_{sharp}(\log D(S)) + E_{blur}(\log(1 - D(G(B))))$$

We combine the multi-scale content and adversarial loss to get the total loss as:

$$L_{total} = L_{content} + \lambda \times L_{adv}$$

We apply adaptive tuning of weights using the ADAM optimizer which uses the gradients derived from back propagation on this loss.

Training

The proposed model is trained using the GoPro dataset. Out of 3,214 pairs of images, we used 2,103 pairs of images for training. We used different augmentation techniques to prevent the problem of overfitting. For example, in geometric transformations, we will randomly flip the patches horizontally or vertically and rotate the patch by 90 degrees. Gaussian random noise is also added to the blurry images. In the process of optimizing the network parameters, we trained the model in a combination of two losses viz., multi-scale content loss and adversarial loss. We are using ADAM optimiser with mini batch size of 2 for training as we are not using batch normalization in our model. The learning rate is fixed as 5×10^{-5} . Total training takes 9×10^5 iterations to converge.

No.	Layer	Weight dimension	Stride
1	conv	32 x 3 x 5 x 5	2
2	conv	64 x 32 x 5 x 5	1
3	conv	64 x 64 x 5 x 5	2
4	conv	128 x 64 x 5 x 5	1
5	conv	128 x 128 x 5 x 5	4
6	conv	256 x 128 x 5 x 5	1
7	conv	256 x 256 x 5 x 5	4
8	conv	512 x 256 x 5 x 5	1
9	conv	512 x 512 x 4 x 4	4
10	fc	512 x 1 x 1 x 1	-
11	sigmoid	-	-

Table 1: Model params. of discriminator

Once we have recovered a sharpened image, we extract the optical flow of the frames to get objects in motion. We represent this extraction process in Figure 6 where we have determine the optical flow of pixels in the first step followed by masking the sharpened image with the blurred image to create a focus on the objects in motion. In order to account to sparseness and patches in flow estimation, we use dilation to cover the entirety of the object. Lastly, we set a threshold on motion which allows us to filter out background motion. This generates a focused and enhanced effect on the moving objects in the image.

Optical flow

Optical flow is defined as the apparent motion of individual pixels on the image plane. It often serves as a good approximation of the true physical motion projected onto the image plane. [4] To estimate optical flow we used opencv's `calcOpticalFlowFarneback` function. Which is an implementation of the Farneback optical flow estimation algorithm. We chose this algorithm as experimentally the Farneback optical flow has been shown to outperform both the Horn-Schunck and Lucas-Kanade optical flow estimation algorithms both in terms of efficiency and accuracy of motion estimation [5]. The parameters of the flow estimation are shown in Table 2. The flow is used to calculate the total magnitude of motion of each pixel. Based on these values the model selects regions to use either the input frames or the frames created by the generator. The ideal threshold to detect motion blur is three pixels. That is is the pixel moves by three or more pixels between consecutive frames then motion blur is likely to be found in the second frame.

No.	Parameters	Value
1	pyramid scale	15
2	levels	5
3	window size	10
4	poly n	5
5	poly sigma	0
6	flags	10

Table 2: Model parameters of Optical Flow

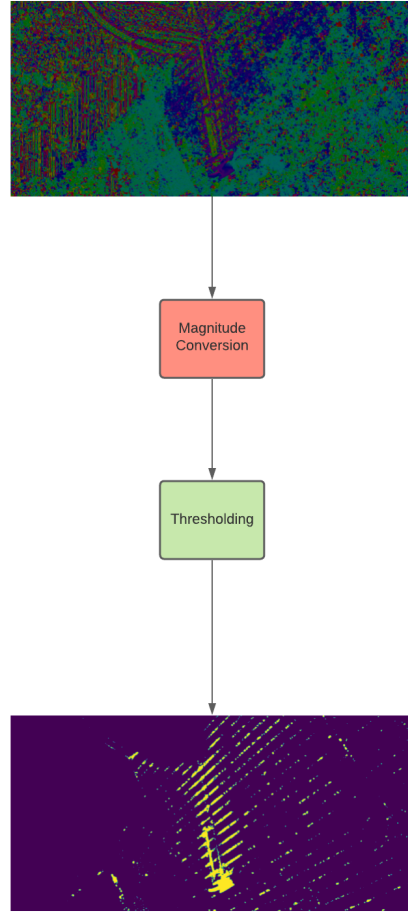


Figure 3: Optical Flow to Mask Conversion

Erode and Dilate

Erosion and dilation are applications of convolutional filtering. Erosion applies a minimum value kernel while dilation applies a max value kernel. For example erosion filtering will return output pixel 0 if the minimum value of all pixels in the neighborhood is 0. The opposite is true for the dilation filter Fig 4.

With figures. and also a table explaining the parameters of erosion and dilation also in a table.

Evaluation Parameters

PSNR

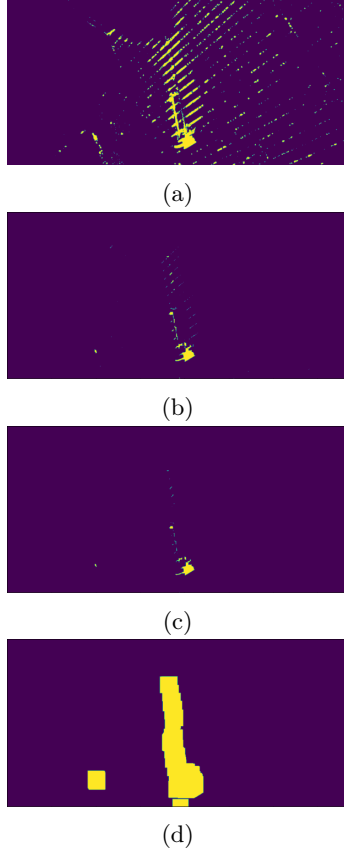


Figure 4: Step wise Erosion and Dilation

No.	Kernel Size	Iterations	Process
1	10	1	Erode
2	3	3	Erode
6	10	10	Dilate

Table 3: Model parameters of Erosion and Dilation

The PSNR block computes the peak signal-to-noise ratio, in decibels, between two images. This ratio is used as a quality measurement between the input blurry image and output sharp image. The higher the PSNR, the better the quality of the output, or reconstructed image.

The mean-square error (MSE) and the peak signal-to-noise ratio (PSNR) are used to compare image reconstruction quality. The MSE represents the cumulative squared error between the reconstructed and the input image, whereas PSNR represents a measure of the peak error. The lower the value of MSE, the lower the error.

SSIM

The Structural Similarity Index (SSIM) metric extracts 3 key features from an image:

- Luminance
- Contrast
- Structure

The comparison between the two images is performed on the basis of these 3 features. In our model, we are using SSIM to compare our output sharp image with the sharp image pair of the blurry input image.

3 Results

We trained a series of datasets with our GAN and used the ResNet architecture to enhance image quality and frame rate.

Effects of using multiple GoPro frames

Given our images we see that the GAN has generated sharper images and has deleted any frames which were "misplaced". Moreover, these frames which were out of place made the footage itself noncontinuous and staggered during playback. As our images were processed through our ResNet the overall quality improved greatly. The qualitative results were frames that had more visual clarity, this can be see between Fig. 5 and Fig. 6 where 8 frames of each set are being compared. Between frames that aren't especially blurry this sharpening was quite noticeable, however the blurriest images showed significant improvement in quality.

We need to compare with these are tables in the first reference paper: Quantitative deblurring performance comparison on the GOPRO dataset. K denotes the scale level

Quantitative comparison on the K"ohler dataset. The dataset has its own evaluation code, thus we report multi-scale SSIM instead of SSIM.



Figure 5: Blurry images before being processed.



Figure 6: Processed result images

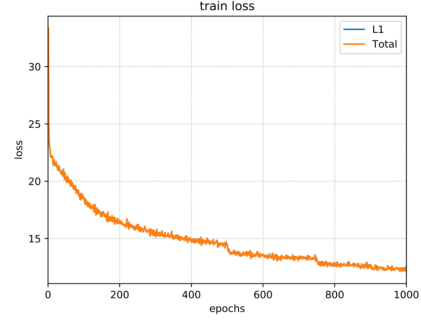


Figure 9: Loss in trained images

The PSNR and SSIM yielded an interesting result with our image set. Throughout the epochs the PSNR and SSIM of the test images followed each others trend for the most part, as seen in Fig. 7. That trend being the amount of PSNR/SSIM values steadily increasing over the epochs in a logarithmic fashion.

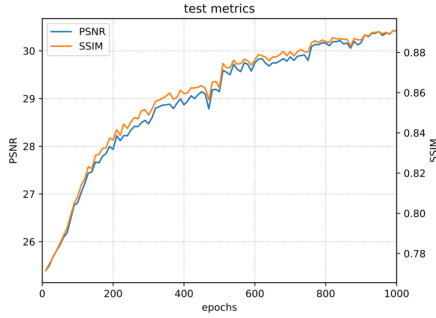


Figure 7: PSNR/SSIM rates vs. Epochs for test images

Another thing to note was the loss in the data set between the test images (Fig. 8) and trained images (Fig. 9). The loss had less variability between epochs in the trained images. A slight difference between sets of images would be the overall loss, which was lower in the trained images as well.



Figure 8: Loss in test images before processing.

4 Discussion

Artifacts generated in non-moving sections of frames of the video. These artifacts can be removed by finding the region which has low optical flow. Thus using the optical flow we can isolate the regions of each frame that are moving fast and causing motion blur and replace only those regions with the output from the sharpening image generator.

To calculate dense optical flow we used opencv's `calcOpticalFlowFarneback` function. This is because the other didn't have a dense option.

Our initial hypothesis was that the output from the generator would be sufficient to generate a sharpened video frame. However the generator couldn't consistently create clear sharpened images as it had a tendency to generate small but noticeable artifacts in the non-moving regions (Fig. 11).

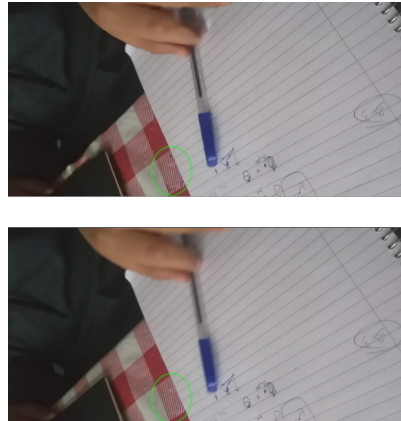


Figure 11: Sample of artifacts generated

The ideal solution would be to tune the parameters and data and retrain the network to prevent it from generating artifacts. A much simpler and elegant solution was to use only the regions required sharpening (Fig. 12).

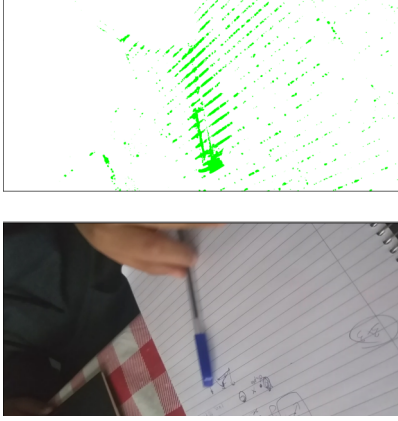


Figure 12: Region of Motion detected by Optical Flow

The second approach used the optical flow between frames to prevent the replacement of pixels that are not moving. This required a threshold of optical flow to decide if a pixel was moving or not. The threshold was set to remove pixels that moved less than speeds that cause observable motion blur.

This approach caused graininess in the frames as the optical flow isn't perfectly accurate at each moving pixel. And there are regions where adjacent alternate pixels crossed and didn't cross the threshold. Which resulted in regions that contained pixels that crossed the threshold as well as pixels that did not cross the threshold(Fig. 13, 14).

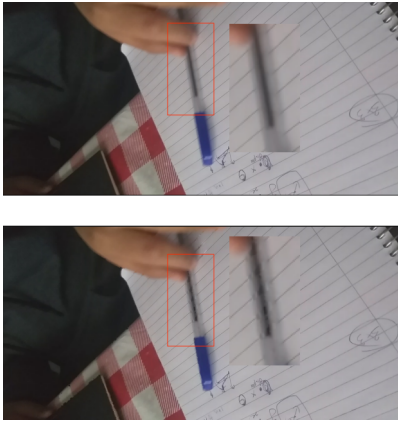


Figure 13: Grainy image caused by improper masking

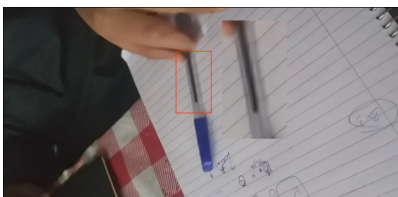


Figure 14: Result of optical flow

To fix this our third approach used the cv functions erode and dilate on the threshold mask(Fig. 15, 16).

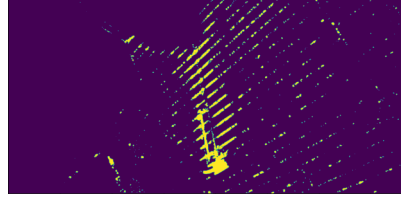


Figure 15: Input mask



Figure 16: Output mask

This resulted in a mask that clearly contained the all the regions that had motion blur. Using this adapted threshold mask we replaced the motion-blurred regions with the output from the sharpened image(Fig. 17, 18). (Fig x5)

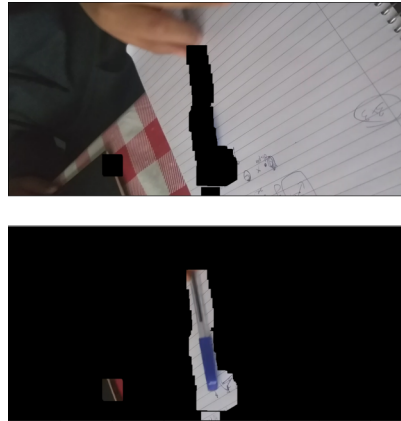


Figure 17: **Top Frame:** Image without motion blur regions. **Bottom Frame:** Mask with sharpened image regions

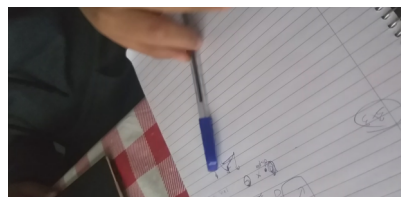


Figure 18: Output frame

5 Conclusions

In this study we have utilized neural networks to deblur video affected by motion blur from multiple sources. Relying on a combination of GAN and ResNet architectures we were able to accomplish this feat. In comparison to past attempts to solve this issue, our approach was much more efficient in terms of speed and efficacy.

Our approach yielded an output that combined deep learning based approaches with traditional algorithms to produce sharpened outputs without any unwanted artifacts.

6 Acknowledgements

Specials thanks to Professor Sathya Ravi, this project wouldn't be possible without his guidance. His course taught us many useful computer vision techniques which we have implemented in this project.

We have also referred many other papers for the improvement of the idea behind the project as well [6, 7, 8, 9, 10, 11].

References

- [1] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [2] Seungjun Nah, Tae Hyun Kim, and Kyoung Mu Lee. Deep multi-scale convolutional neural network for dynamic scene deblurring, 2018.
- [3] Tae Hyun Kim and Kyoung Mu Lee. Generalized video deblurring for dynamic scenes, 2015.
- [4] Pavan Turaga, Rama Chellappa, and Ashok Veeraraghavan. Advances in video-based human activity analysis: Challenges and approaches. In Marvin V. Zelkowitz, editor, *Advances in Computers*, volume 80 of *Advances in Computers*, pages 237–290. Elsevier, 2010.
- [5] Neeta Nemade and V. V. Gohokar. Comparative performance analysis of optical flow algorithms for anomaly detection. *SSRN Electronic Journal*, 05 2019.
- [6] Shuochen Su, Mauricio Delbracio, Jue Wang, Guillermo Sapiro, Wolfgang Heidrich, and Oliver Wang. Deep video deblurring for hand-held cameras. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 237–246, 2017.
- [7] Sunghyun Cho, Jue Wang, and Seungyong Lee. Video deblurring for hand-held cameras using patch-based synthesis. *ACM Trans. Graph.*, 31(4), jul 2012.
- [8] Ayan Chakrabarti. A neural approach to blind motion deblurring, 2016.
- [9] Xintian Mao, Yiming Liu, Wei Shen, Qingli Li, and Yan Wang. Deep residual fourier transformation for single image deblurring, 2021.
- [10] Seungjun Nah. Deepdeblur-pytorch. <https://github.com/SeungjunNah/DeepDeblur-PyTorch>, 2021.
- [11] Kuldeep Purohit and A. N. Rajagopalan. Region-adaptive dense network for efficient motion deblurring, 2019.